Step 3 - Design Deep Dive

In this section, we examine the internal design of critical Google Drive components in detail.

The major topics covered are:

- Block servers
- Metadata database
- Upload flow
- Download flow
- Notification service
- Storage optimization
- Failure handling

Block Servers

For large files that are modified frequently, uploading the entire file every time is extremely inefficient.

Problems with full file uploads:

- High bandwidth consumption
- Slow synchronization
- Increased cloud storage traffic
- Poor user experience

To solve these problems, the system introduces two major optimizations:

1. Delta Sync
2. Compression

Delta Sync

Delta sync ensures that only modified portions of a file are synchronized instead of transferring the entire file.

This dramatically reduces:

- Upload bandwidth
- Synchronization time
- Network traffic

How Delta Sync Works

Suppose a large file contains multiple blocks.

If only a few blocks change:

- Unchanged blocks are ignored
- Only modified blocks are uploaded

This minimizes redundant data transfer.

Delta Sync Example

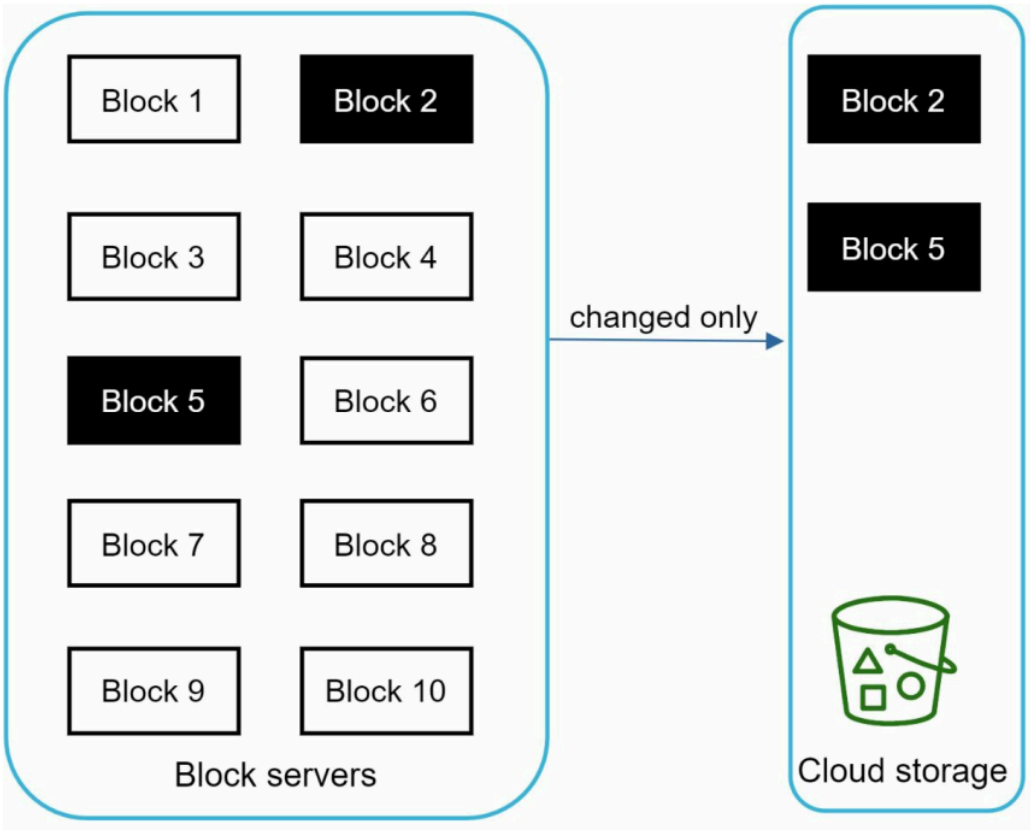

Figure demonstrates delta synchronization.

In the figure:

- A file is divided into multiple blocks
- Only **block 2** and **block 5** are modified
- Only those modified blocks are uploaded to cloud storage

Benefits:

- Faster synchronization
- Reduced bandwidth usage
- Efficient large file updates

Compression

Before blocks are uploaded:

- Each block is compressed

Compression significantly reduces data size.

Different file types use different compression algorithms.

Examples:

File Type	Compression Algorithm
Text files	gzip, bzip2
Images	Image-specific compression
Videos	Video codecs

Benefits:

- Lower storage cost
- Faster uploads
- Reduced network traffic

Responsibilities of Block Servers

Block servers perform the heavy processing tasks for uploads.

Their responsibilities include:

1. File splitting
2. Compression
3. Encryption
4. Block uploading

File Splitting

A file is divided into smaller independent blocks.

Advantages:

- Parallel uploads
- Resumable uploads
- Delta synchronization
- Better scalability

Each block:

- Has a unique hash
- Is independently addressable

Compression Stage

After splitting:

- Each block is compressed individually

This reduces:

- File transfer size
- Cloud storage space consumption

Encryption Stage

Security is critical for cloud storage systems.

Before blocks are uploaded:

- Each block is encrypted

Benefits:

- Protects user privacy

- Prevents unauthorized access
- Ensures secure storage

Encryption applies:

- During transmission
- During storage

Uploading Blocks to Cloud Storage

Once compressed and encrypted:

- Blocks are uploaded to distributed cloud object storage

Examples:

- Amazon S3
- Google Cloud Storage

Block Server Workflow

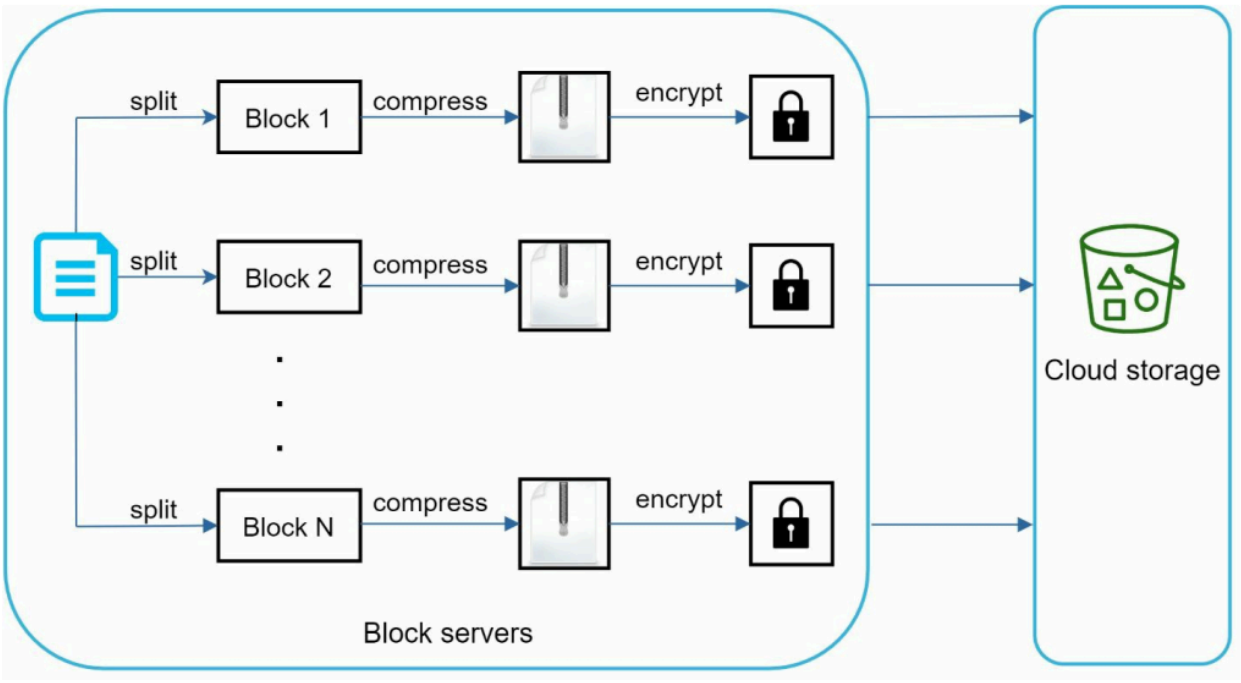

Figure illustrates the complete workflow of block servers when uploading a new file.

The process includes:

1. Splitting file into blocks
2. Compressing each block
3. Encrypting each block
4. Uploading blocks to cloud storage

This architecture improves:

- Efficiency
- Scalability
- Reliability
- Security

Advantages of Block Servers

Feature	Benefit

Delta sync	Reduced bandwidth
Compression	Lower storage cost
Encryption	Data security
Block uploads	Faster synchronization
Parallel processing	Improved scalability

High Consistency Requirement

A distributed file synchronization system requires:

Strong Consistency

It is unacceptable for:

- Different devices to show different file versions simultaneously

For example:

- User edits a document on laptop
- Mobile app still shows old version

This creates inconsistency and poor user experience.

Strong Consistency Requirements

The system guarantees:

1. Metadata consistency across replicas
2. Consistency between cache and database
3. Correct synchronization ordering

Cache Consistency Problem

Most distributed cache systems use:

Eventual Consistency

Meaning:

- Replicas may temporarily contain different values

This behavior is problematic for file synchronization systems.

Strong Consistency Strategy

To achieve strong consistency, the system enforces:

1. Cache and Database Synchronization

Whenever database updates occur:

- Corresponding cache entries are invalidated or updated immediately

This ensures:

Cache = Database

2. Replica Consistency

All cache replicas must maintain synchronized state.

This prevents:

- Stale reads
- Incorrect file versions
- Synchronization conflicts

Why Relational Databases?

The system chooses relational databases because they support:

ACID Properties

ACID stands for:

Property	Meaning
Atomicity	Operations complete fully or not at all
Consistency	Database remains valid
Isolation	Concurrent operations do not interfere
Durability	Committed data survives failures

These guarantees are essential for:

- File synchronization
- Version management
- Conflict handling

ACID Consistency

Relational databases naturally provide strong consistency guarantees.

This makes them ideal for:

- Metadata management
- File revision tracking
- Permission systems

Examples:

- MySQL
- PostgreSQL

Why Not NoSQL?

Many NoSQL systems prioritize:

- Scalability
- Availability

But they often sacrifice:

- Strong consistency

NoSQL databases generally use:

- Eventual consistency models

To support ACID-like behavior in NoSQL:

- Additional synchronization logic must be implemented manually

This increases:

- System complexity
- Development effort
- Risk of synchronization bugs

Metadata Database Responsibilities

The metadata database stores:

- User information
- File metadata
- File versions
- Block references
- Sharing permissions
- Sync state

Important:

- Actual file content is NOT stored here
- Only metadata is stored

Summary of Deep Dive Concepts

Component	Purpose
Block servers	Process uploads efficiently
Delta sync	Upload only changed blocks
Compression	Reduce bandwidth/storage
Encryption	Secure file storage
Cloud storage	Durable object storage

Relational DB	Strong consistency
Cache invalidation	Maintain synchronization

Key Takeaways

The Google Drive architecture achieves:

- Efficient synchronization
- Strong consistency
- Secure storage
- Reduced bandwidth usage
- High scalability
- Reliable distributed file management

Using:

- Block-level storage
- Delta synchronization
- Compression
- Encryption
- ACID-compliant metadata databases